ChatGPT

Why AI Results Are Non-Deterministic and How Statistics Blends into AI

Executive summary

AI outputs are non-deterministic for several different reasons, and they do not all come from the same place. Some variability is deliberate, such as stochastic training, random initialisation, dropout, data shuffling, and token sampling at generation time. Some variability is numerical or systems-level, such as asynchronous GPU execution, different parallel reduction orders, and floating-point non-associativity. Some variability is operational, such as changing datasets, train/validation splits, hosted model updates, or external tools like web search and retrieval APIs that can return different information over time. Framework documentation from PyTorch and TensorFlow explicitly warns that exact reproducibility is not guaranteed across releases, platforms, CPUs and GPUs, even with fixed seeds, while OpenAI documents that hosted model outputs are non-deterministic by default and only "mostly" reproducible under fixed seeds and matching backend fingerprints. [1]

The line between "traditional statistics" and "AI" is best understood as a continuum, not a clean mathematical boundary. Classical statistics usually centres explicit assumptions, parameter estimation, hypothesis testing, causal interpretation, and uncertainty quantification. Modern AI more often prioritises out-of-sample prediction, generation, representation learning, and empirical performance on held-out data, often using flexible non-parametric or over-parameterised function classes. Breiman (2001) framed a related contrast as "data modelling" versus "algorithmic modelling", while Shmueli (2010) distinguished explanation from prediction; Pearl (2009) further separated ordinary statistical description from causal analysis, and Gelman et al. (2020) emphasised uncertainty-aware Bayesian workflow as an inferential tradition within the broader modelling landscape. [2]

In practice, a methodology looks more like traditional statistics when the inferential target is a parameter, effect, or scientifically interpretable relationship under explicit assumptions. It looks more like an AI model when the main objective is predictive or generative performance and the model is evaluated chiefly by empirical accuracy, loss, or benchmark performance. The intermediate zone is large: GLMs, Bayesian models, decision trees, ensembles, SVMs, and probabilistic graphical models all borrow from both traditions. [3]

If reproducibility matters, the most effective controls are simple but strict: seed every RNG you use; materialise and store data splits; force deterministic kernels where the framework supports them; pin software, hardware, and precision settings; log model IDs, hyperparameters, artefacts, and backend fingerprints; and freeze external tool dependencies or record their responses. When exact bitwise determinism is impossible, aim instead for strong procedural reproducibility and stable aggregate performance. [4]

Why AI outputs vary

A useful starting point is to separate **mathematical determinism** from **system determinism**. A fixed function with fixed inputs is mathematically deterministic. But real AI systems are pipelines: they include random number generators, approximate optimisation, hardware-specific kernels, floating-

point arithmetic, sampling-based decoders, changing data, and sometimes external tools. Many “non-deterministic” AI behaviours are therefore better described as **state-sensitive** or **environment-sensitive** rather than mysterious. PyTorch and TensorFlow both document this explicitly, and OpenAI states that hosted text generation is non-deterministic by default. [5]

Source of variation	What changes	Simple concrete example	Can you reduce it?
Model architecture	Some architectures include stochastic components during training, such as dropout (Srivastava et al., 2014), or use input-dependent routing such as mixture-of-experts layers (Fedus et al., 2021/2022).	The same minibatch passed through a training-time dropout layer gets a different mask on different passes.	Often yes for inference, partly for training.
Stochastic training	Initial weights, minibatch order, data augmentation, and optimiser noise alter the path through parameter space.	Two neural nets trained on the same data but with different seeds land in different local solutions.	Partly, by fixing seeds and the input order.
Random seeds	Pseudo-random generators produce different sequences unless seeded consistently.	`random.random()` or `np.random.default_rng()` gives different draws if no fixed seed is used.	Yes, within a fixed software/hardware environment.
Hardware and parallelism	Parallel kernels and asynchronous execution can change operation order.	GPU reductions can sum values in different orders across runs or kernels.	Partly, with deterministic kernels and pinned hardware.
Floating-point arithmetic	Floating-point addition and multiplication are not associative, so order matters.	Batched matrix multiplication may differ slightly from doing the same multiplications one by one.	Only partly; numerical differences can remain.
Sampling and decoding	Token choice may be greedy, beam-based, or sampled from a distribution.	Greedy decoding always picks the most likely token; top-*p* sampling draws one at random from a truncated distribution.	Yes, by using greedy decoding or fixed seeds where supported.

Source of variation	What changes	Simple concrete example	Can you reduce it?
Data variability	Different samples, splits, labels, preprocessing rules, or ordering produce different fitted models.	A decision tree trained after a small data change can become a different tree.	Yes, by versioning data and saving split indices.
Model updates	Hosted providers may change weights, routing, infrastructure, or numerical settings.	The same API call returns a different answer after the backend fingerprint changes.	Only partly, if version pinning and fingerprints are exposed.
External APIs and tools	Search, retrieval, or live databases may return different information over time.	A web-search-enabled model answers differently today because the underlying web pages changed.	Yes, by freezing or logging tool responses.

This table synthesises framework documentation and seminal papers on dropout, mixture-of-experts routing, token sampling, and hosted-model reproducibility. [6]

A few of these sources deserve extra precision. PyTorch warns that complete reproducibility is not guaranteed across releases, commits, platforms, or even CPU versus GPU runs with identical seeds. TensorFlow similarly notes that deterministic execution requires explicitly enabling deterministic ops, and that otherwise asynchronous threads, especially on GPUs, may change the order in which floating-point values are added. NumPy also notes that its `Generator` offers **no version compatibility guarantee** for the produced bit stream as algorithms evolve. [7]

Hosted LLMs add another layer. OpenAI documents that responses are non-deterministic by default; fixed `seed` values and matching parameters can make outputs **mostly** deterministic, but backend changes captured by `system_fingerprint` can still alter results, and determinism is not guaranteed even when the fingerprint matches. If tool use is enabled, outputs can also change because the tool itself is live. [8]

Short code snippets illustrating deterministic versus non-deterministic behaviour

A pure deterministic function returns the same output every time for the same input.

```
def f(x):
    return 2 * x + 1

print(f(4))   # 9
print(f(4))   # 9
print(f(4))   # 9
```

Unseeded pseudo-randomness does not.

```
import random

print([random.random() for _ in range(3)])  # different each run
print([random.random() for _ in range(3)])  # different again
```

But fixed seeds can make pseudo-randomness reproducible within the same environment.

```
import random
import numpy as np

random.seed(42)
print([random.random() for _ in range(3)])

random.seed(42)
print([random.random() for _ in range(3)])  # identical

rng = np.random.default_rng(seed=42)
a = rng.random((2, 2))

rng = np.random.default_rng(seed=42)
b = rng.random((2, 2))

print(np.allclose(a, b))  # True
```

The Python and NumPy documentation explicitly describe these generators as pseudo-random and seedable, while also warning that reproducibility can depend on threading and versioning details. [9]

Training-time stochasticity is easy to illustrate with dropout.

```
# Pseudocode / PyTorch-like behaviour
model.train()
y1 = model(x)   # dropout mask A
y2 = model(x)   # dropout mask B, usually different

model.eval()
y3 = model(x)   # dropout disabled
y4 = model(x)   # same as y3 if everything else is fixed
```

Srivastava et al. (2014) describe dropout as **randomly dropping units** during training, which is useful for regularisation but a direct source of within-training variability. [10]

Floating-point non-associativity means two mathematically equivalent computations can differ numerically.

```
x = 1e20
y = -1e20
z = 3.14
```

```
print((x + y) + z)   # 3.14
print(x + (y + z))   # 0.0
```

That same principle explains why batched and non-batched tensor operations can disagree at the bit level. PyTorch documents that `(A @ B)[0]` is not guaranteed to be bitwise identical to `A[0] @ B[0]`, even though the mathematics is the same on paper. [11]

For language models, decoding strategy is often the most visible source of output variation.

```
# Pseudocode for next-token decoding
probs = model.next_token_distribution(context)

# More deterministic
token = argmax(probs)                # greedy decoding

# Less deterministic
token = sample(top_p_filter(probs)) # stochastic sampling
```

Hugging Face documents that greedy decoding selects the most likely next token, whereas multinomial sampling draws randomly from the token distribution; Holtzman et al. (2020) show that decoding strategy alone can substantially change generated text quality even with the same underlying model. [12]

Finally, this is the standard pattern for **best-effort** reproducibility in a modern deep-learning stack.

```
import os, random, numpy as np, torch

seed = 0
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)

torch.use_deterministic_algorithms(True)
torch.backends.cudnn.benchmark = False
torch.backends.cudnn.deterministic = True
```

This helps, but PyTorch still warns that full reproducibility is not guaranteed across releases, platforms, or CPU/GPU boundaries. TensorFlow offers a similar switch via `tf.config.experimental.enable_op_determinism()`. [13]

Where traditional statistics ends and AI begins

The strongest answer is that there is **no single sharp cut-off**. The difference is chiefly one of **modelling culture, objective function, evaluation practice, and inferential ambition**. NIST's description of classical analysis emphasises imposing a model and then estimating or testing its parameters. Breiman (2001) contrasted that with "algorithmic modelling", where the emphasis is predictive performance rather than faithful specification of a generative data model. Shmueli (2010) made a similar distinction

between explanatory modelling and predictive modelling, and Pearl (2009) argued that causal analysis requires additional language and machinery beyond ordinary statistical parameter inference. Gelman et al. (2020) place Bayesian analysis within an uncertainty-centred inferential workflow rather than a purely predictive one. [14]

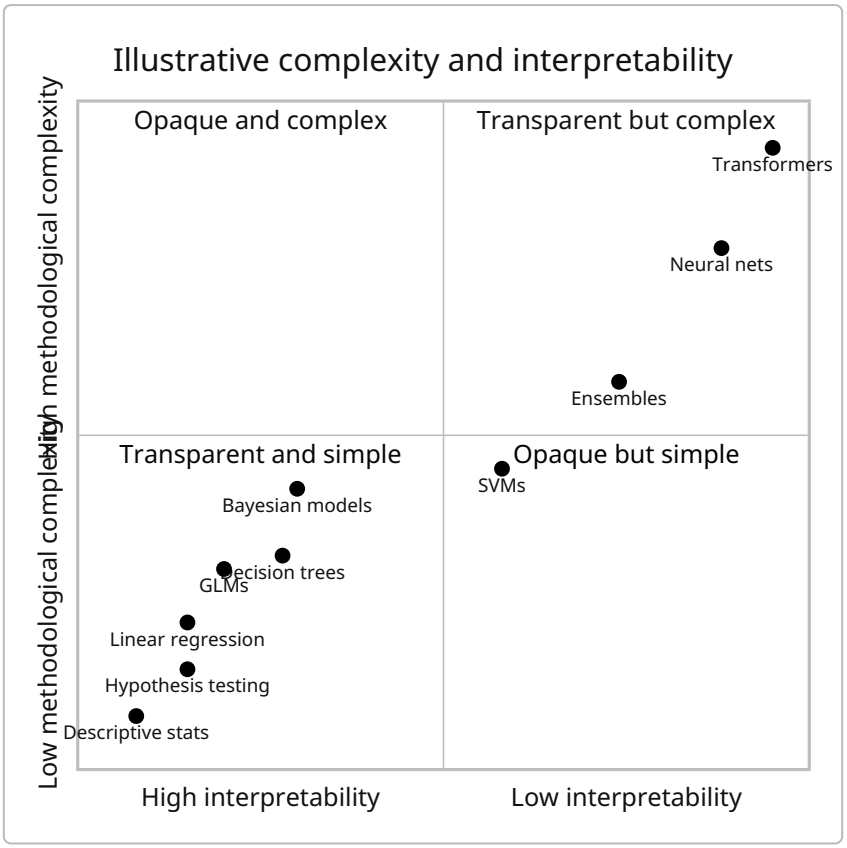
A practical rule is therefore this. If your central question is, "What effect does *X* have on *Y*, under what assumptions, and with what uncertainty?", you are mostly in traditional statistics. If your central question is, "Given inputs, what output gives the best prediction, classification, ranking, or generation on new data?", you are mostly in AI or statistical learning. If both matter, you are in the middle of the continuum. [15]

Dimension	Traditional statistics tendency	AI model tendency	What to look for in practice
Assumptions	Explicit distributional or structural assumptions are central.	Flexible function classes are often preferred; assumptions may be weaker or less transparent.	Ask whether validity depends on a specified stochastic or causal model.
Interpretability	Parameter interpretation is often first-class.	Interpretation may be secondary, post-hoc, or local rather than global.	Are coefficients, odds ratios, or effects the deliverable, or just predictive accuracy?
Typical data size	Often designed to work well with small-to-moderate, carefully collected samples.	Often benefits from medium-to-large, high-dimensional, weakly structured data.	If performance improves mainly by scale and representation learning, the method leans AI.
Objective	Estimation, testing, explanation, or causal effect measurement.	Prediction, ranking, control, generation, or representation learning.	Is the target a parameter/effect or an output?
Inference versus prediction	Inferential validity is central.	Held-out performance is central.	Are you reporting *p*-values/posteriors, or test loss/AUROC/BLEU?
Parametric versus non-parametric	Parametric and semi-parametric methods are common.	Non-parametric, kernel, tree-based, and over-parameterised models are common.	How much structure is specified before seeing the data?
Causality focus	Frequently explicit.	Often absent unless causal ML is deliberately used.	Would the model still be useful if correlations shift under intervention?

Dimension	Traditional statistics tendency	AI model tendency	What to look for in practice
Uncertainty quantification	Confidence intervals, standard errors, and posterior distributions are native outputs.	Uncertainty often needs explicit extra machinery, calibration, or ensembles.	Does uncertainty come out “for free”, or must it be engineered separately?
Optimisation profile	Closed-form or convex optimisation is common.	Large-scale non-convex optimisation is common in deep learning.	Can you solve it analytically or with guaranteed convex solvers?
Evaluation culture	Assumption checks, residuals, identification, sensitivity analysis.	Validation loss, benchmark scores, ablations, and scalability.	What counts as a successful result?

This comparison is a synthesis of NIST’s classical/Bayesian framing, Breiman’s data-versus-algorithmic modelling, Shmueli’s explanation-versus-prediction distinction, Pearl’s separation of causal from ordinary statistical analysis, and Gelman et al.’s uncertainty-centred Bayesian workflow. [14]

The main implication is methodological rather than semantic: many methods commonly called “AI” are still statistical models, and many statistical models are powerful predictive learners. The classification should therefore be driven by **purpose**, not branding. A random forest used to maximise hold-out accuracy on churn prediction behaves like AI; a hierarchical regression used to estimate policy effects with confidence intervals behaves like traditional statistics; a Bayesian neural network sits in between. [16]



This chart is intentionally illustrative rather than empirical; it summarises the trade-offs discussed in the taxonomy below. [17]

A progressive taxonomy from statistics to AI

The ordering below moves from simpler summarisation and inference toward more flexible prediction and generation. The "determinism likelihood" column refers to **common implementations in normal use**, not to mathematical necessity. A feed-forward model with frozen weights and greedy decoding can be deterministic; a simple tree trained in a pipeline with random feature subsampling may not be. [18]

Method	Short description	Toy example	Strengths	Weaknesses	Typical determinism likelihood
Descriptive statistics	Summaries such as mean, median, variance, quartiles, and plots; they reduce data without fitting a predictive mechanism (NIST/SEMATECH, 2012).	Summarise weekly shop sales: mean basket value, median items per basket, standard deviation.	Transparent, cheap, essential first step.	No counterfactual, inferential, or predictive mechanism by itself.	**Very high** once data are fixed.

Method	Short description	Toy example	Strengths	Weaknesses	Typical determinism likelihood
Hypothesis testing	Formal tests of claims about population parameters using test statistics, significance levels, and power (NIST/SEMATECH, 2012).	Test whether average battery life exceeds 10 hours.	Clear inferential target; explicit error rates.	Sensitive to assumptions, sample size, and question formulation; significance is not practical importance.	**High** once data and test choice are fixed.
Linear regression	Ordinary least squares fits a linear combination of features by minimising residual sum of squares (Scikit-learn Developers, 2026; NIST/SEMATECH, 2012).	Predict house price from floor area and age.	Interpretable coefficients, fast fitting, strong baseline.	Assumes linear form; sensitive to misspecification, outliers, and collinearity.	**High** in standard convex implementations.
Generalised linear models	Extend linear models to non-Gaussian outcomes using link functions and exponential-family likelihoods (Nelder & Wedderburn, 1972).	Logistic regression for pass/fail; Poisson regression for defect counts.	Keeps interpretability while handling binary, count, and rate outcomes.	Still assumption-heavy; limited on complex nonlinear interactions unless extended.	**High** in standard fitting routines.
Bayesian methods	Use priors plus likelihood to produce posterior distributions, emphasising uncertainty and workflow-based model checking (Gelman et al., 2020).	Estimate a drug success rate with prior clinical knowledge and observed successes/failures.	Native uncertainty quantification; priors encode knowledge; strong for hierarchical data.	Computationally heavier; prior sensitivity; approximate inference can introduce Monte Carlo variation.	**Medium**: exact formulas can be deterministic, MCMC and variational methods are often not fully so.

Method	Short description	Toy example	Strengths	Weaknesses	Typical determinism likelihood
Decision trees	Learn decision rules by recursive partitioning; non-parametric and piecewise-constant (Quinlan, 1986; Scikit-learn Developers, 2026).	Approve a loan if income is high and arrears are low.	Readable rules; captures nonlinearity and interactions; handles mixed feature types well.	Unstable: small data changes can yield a different tree; poor extrapolation.	**Medium to high** if randomness is disabled and ties are controlled; otherwise **medium**.
Ensemble methods	Combine many learners. Bagging and random forests average over bootstrap-based trees (Breiman, 1996; Breiman, 2001); boosting adds weak learners stagewise to reduce loss (Friedman, 2001).	Predict customer churn from many behavioural features using hundreds of trees.	Strong tabular performance; reduces variance; captures complex interactions.	Less interpretable than a single tree; more tuning; often stochastic by design.	**Medium** with fixed seeds and frozen pipeline; otherwise **low to medium**.
Support vector machines	Large-margin methods; with kernels they create nonlinear decision boundaries in high-dimensional feature spaces (Cortes & Vapnik, 1995).	Classify spam versus non-spam from sparse word counts.	Strong in high-dimensional settings; effective when features outnumber samples.	Kernel SVMs scale poorly to very large datasets; tuning is delicate.	**High** in many standard solver settings, though probability estimation and tie handling can add complexity.

Method	Short description	Toy example	Strengths	Weaknesses	Typical determinism likelihood
Neural networks	Multi-layer differentiable function approximators trained with backpropagation; deep learning stacks many processing layers to learn representations (Rumelhart et al., 1986; LeCun, Bengio & Hinton, 2015).	Classify handwritten digits or predict energy demand from many sensor inputs.	Extremely expressive; learns representations directly from data; dominant in unstructured domains.	Less interpretable; training is data- and compute-intensive; stochastic optimisation and regularisation introduce variability.	**Low** in ordinary training pipelines.
Probabilistic graphical models	Graphs represent variables and conditional dependences; Bayesian networks can also encode causal structure (Pearl, 2014).	Diagnose disease from symptoms, tests, and latent causes.	Structured reasoning under uncertainty; interpretable dependencies; good with missing data.	Structure learning and inference can be hard; quality depends on graph design.	**Medium to high** with exact inference; **medium** with approximate inference.
Deep generative models	Models such as VAEs learn latent-variable generative structure (Kingma & Welling, 2014), while GANs train generator and discriminator adversarially (Goodfellow et al., 2014).	Learn a latent space of faces or generate synthetic images.	Generation, compression, simulation, and representation learning.	Training can be unstable; evaluation is difficult; outputs usually sampled, not fixed.	**Very low to low** in typical training-and-generation workflows.

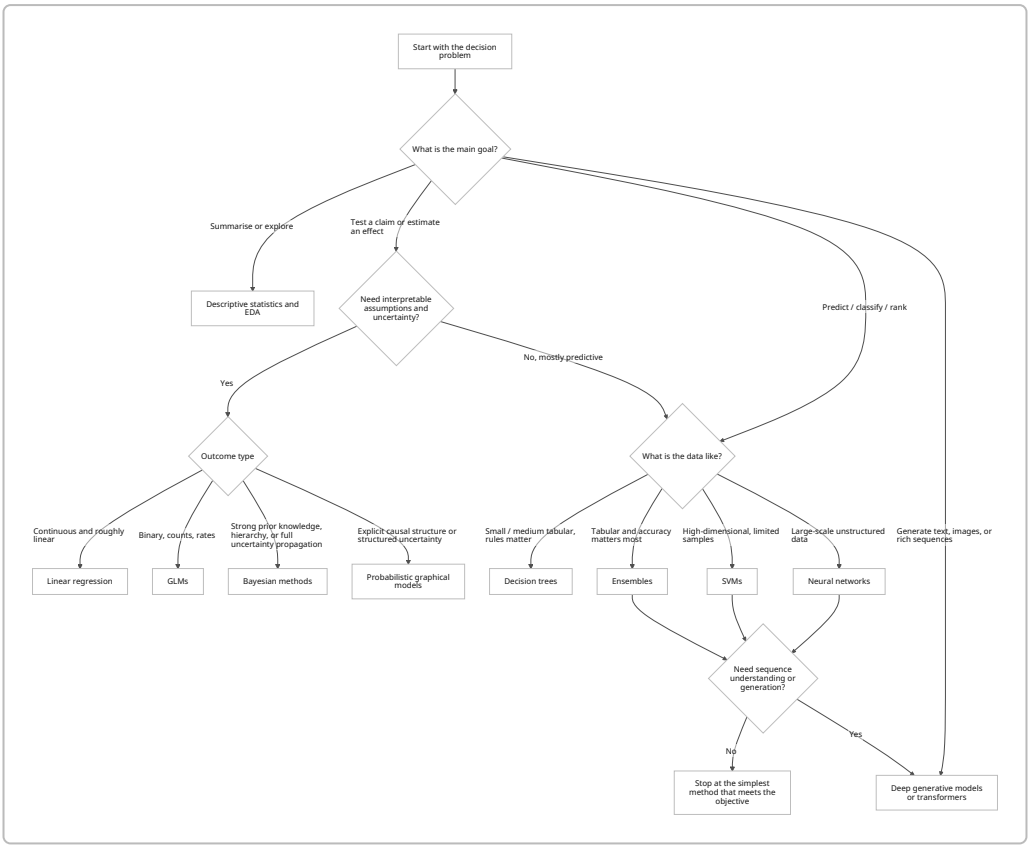
Method	Short description	Toy example	Strengths	Weaknesses	Typical determinism likelihood
Transformers	Attention-based sequence models that dispense with recurrence and convolutions in favour of attention mechanisms (Vaswani et al., 2017); sparse MoE variants route different inputs through different parameters (Fedus et al., 2021/2022).	Next-token prediction, summarisation, translation, or code generation from text prompts.	State-of-the-art for many sequence and multimodal tasks; highly scalable; strong transfer learning.	Compute-heavy, opaque, sensitive to decoding strategy and serving stack; system-level nondeterminism is common.	**Low** at end-to-end system level; **medium** only under very controlled greedy inference.

The method descriptions above rest on official documentation and seminal papers: NIST/SEMATECH for descriptive statistics and testing; Nelder and Wedderburn (1972) for GLMs; Quinlan (1986) for decision trees; Breiman (1996, 2001) and Friedman (2001) for ensembles; Cortes and Vapnik (1995) for SVMs; Rumelhart et al. (1986) and LeCun et al. (2015) for neural networks; Pearl (2014) for graphical models; Kingma and Welling (2014), Goodfellow et al. (2014), and Vaswani et al. (2017) for modern generative and transformer models. Determinism labels are an applied synthesis based on these algorithm families plus modern framework reproducibility notes. [19]

Two interpretive cautions matter. First, several of these methods are simultaneously “statistical” and “AI”: SVMs, trees, ensembles, and Bayesian models are not cleanly on one side. Second, model family and system behaviour are different issues. A linear regression implemented in a batch scoring pipeline is usually deterministic. A transformer served behind a hosted API with tool use, dynamic routing, and live web search can vary even if the underlying mathematical architecture is fixed. [20]

Practical guidance for method choice and reproducibility

Method choice should start from the scientific or operational target rather than from model fashion. If the aim is summarisation, use descriptive statistics. If the aim is testing or effect estimation under assumptions, use classical or Bayesian statistical models. If the main objective is predictive performance on structured tabular data, trees and ensembles are often strong baselines. If the data are high-dimensional but not huge, SVMs can still be excellent. If the data are large and unstructured, neural networks and transformers become increasingly attractive. If uncertainty structure or causal structure is intrinsic to the problem, Bayesian methods or probabilistic graphical models deserve explicit consideration. This way of choosing methods follows the explanation-versus-prediction distinction of Shmueli (2010), the classical/Bayesian framing in NIST, and the predictive emphasis described by Breiman (2001). [21]



Reproducibility control should be treated as a design requirement, not a cleanup step. The table below collects the highest-value controls from framework documentation and production tooling guidance. [22]

Control	What to do	Why it matters	Important caveat
Seed every RNG	Set Python, NumPy, framework, dataloader, and API seeds explicitly.	Prevents hidden shifts in pseudo-random sequences.	Same seed does **not** guarantee cross-platform equality.
Force deterministic kernels where possible	In PyTorch, use deterministic algorithms and disable cuDNN benchmarking; in TensorFlow, enable op determinism.	Reduces kernel-level nondeterminism from parallel execution and algorithm selection.	Usually slower.
Save data splits and ordering	Materialise train/validation/ test indices and data-order rules.	Prevents “same dataset, different split” drift.	Especially important for trees, ensembles, and SGD-based models.
Version data and models	Use DVC or equivalent to bind code, data, and model artefacts together.	Lets you reconstruct the exact experiment state.	Requires team discipline, not just tooling.

Control	What to do	Why it matters	Important caveat
Log experiment metadata	Use MLflow or equivalent for parameters, code versions, metrics, artefacts, and model IDs.	Makes reruns auditable and comparable.	Logging after the fact is too late.
Freeze environment	Pin library versions, CUDA/cuDNN, precision settings, and container image.	Framework and numerical differences can change results.	NumPy itself warns bit streams may change across versions.
Log hardware and precision	Record CPU/GPU type, mixed precision, TF32/FP16/BF16 flags.	Floating-point and hardware paths affect results.	“Same code” is not enough if hardware differs.
Control decoding	For stable generation, prefer greedy or beam search; if sampling is needed, log `temperature`, `top_k`, `top_p`, `num_beams`, and seed.	Decoding can change outputs as much as the model can.	Greedy is more stable, but often less diverse.
Track hosted backend identity	Record model version, checkpoint hash, or backend fingerprint.	Hosted providers can update models or infrastructure.	With opaque APIs, exact freezing may be impossible.
Freeze or log external tool calls	Save retrieved documents, search results, and API responses used at inference time.	Tool-enabled systems can change because the world changed.	This matters even if the model itself is stable.

These recommendations come directly from Python, NumPy, PyTorch, TensorFlow, Hugging Face, OpenAI, MLflow, DVC, and Docker documentation. [23]

The most robust conclusion is therefore practical. If you need **scientific interpretation**, start with the simplest statistical model whose assumptions you can defend. If you need **prediction**, start with a strong but controllable baseline such as linear models, GLMs, trees, or ensembles before escalating to deep learning. If you need **generation or rich sequence modelling**, transformers and deep generative models are often appropriate, but their outputs should be treated as products of a whole system, not just of a frozen function. And if exact repeatability matters, ask not only, “What model did I run?” but also, “What data, what split, what seed, what kernel, what hardware, what decoding rule, and what external tools were live at the time?” [24]

Source URLs

```
NIST/SEMATECH e-Handbook of Statistical Methods
https://www.itl.nist.gov/div898/handbook/

NIST EDA versus classical and Bayesian
```

https://www.itl.nist.gov/div898/handbook/eda/section1/eda12.htm

NIST summary analysis versus EDA
https://www.itl.nist.gov/div898/handbook/eda/section1/eda13.htm

NIST quantitative techniques and hypothesis tests
https://www.itl.nist.gov/div898/handbook/eda/section3/eda35.htm

NIST linear least squares regression
https://www.itl.nist.gov/div898/handbook/pmd/section1/pmd141.htm

Breiman 2001 Statistical Modelling The Two Cultures
https://projecteuclid.org/journals/statistical-science/volume-16/issue-3/Statistical-Modeling--The-Two-Cultures-with-comments-and-a/10.1214/ss/1009213726.full

Shmueli 2010 To Explain or to Predict
https://www.stat.berkeley.edu/~aldous/157/Papers/shmueli.pdf

Pearl 2009 Causal Inference in Statistics An Overview
https://www.cs.princeton.edu/courses/archive/fall09/cos597A/papers/Pearl2009.pdf

Gelman et al. 2020 Bayesian Workflow
https://arxiv.org/abs/2011.01808

Nelder and Wedderburn 1972 Generalized Linear Models
https://jhanley.biostat.mcgill.ca/bios601/Likelihood/NelderWedderburn1972.pdf

Quinlan 1986 Induction of Decision Trees
https://hunch.net/~coms-4771/quinlan.pdf

Breiman 1996 Bagging Predictors
https://www.stat.berkeley.edu/~breiman/bagging.pdf

Breiman 2001 Random Forests
https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf

Friedman 2001 Greedy Function Approximation A Gradient Boosting Machine
https://www.cse.cuhk.edu.hk/irwin.king/_media/presentations/2001_greedy_function_approximation_a_gradient_boosting_machine.pdf

Cortes and Vapnik 1995 Support-Vector Networks
https://link.springer.com/article/10.1007/BF00994018

Rumelhart Hinton and Williams 1986 Learning Representations by Back-Propagating Errors
https://www.nature.com/articles/323533a0

LeCun Bengio and Hinton 2015 Deep Learning
https://www.cs.toronto.edu/~hinton/absps/NatureDeepReview.pdf

Pearl 2014 Graphical Models for Probabilistic and Causal Reasoning
https://ftp.cs.ucla.edu/pub/stat_ser/r236-3ed.pdf

Kingma and Welling 2014 Auto-Encoding Variational Bayes
https://arxiv.org/abs/1312.6114

Goodfellow et al. 2014 Generative Adversarial Nets
https://papers.neurips.cc/paper/5423-generative-adversarial-nets.pdf

Vaswani et al. 2017 Attention Is All You Need
https://arxiv.org/abs/1706.03762

Fedus Zoph and Shazeer 2021/2022 Switch Transformers
https://arxiv.org/abs/2101.03961

Holtzman et al. 2020 The Curious Case of Neural Text Degeneration
https://arxiv.org/abs/1904.09751

PyTorch reproducibility notes
https://docs.pytorch.org/docs/stable/notes/randomness.html

PyTorch numerical accuracy notes
https://docs.pytorch.org/docs/stable/notes/numerical_accuracy.html

TensorFlow deterministic ops
https://www.tensorflow.org/api_docs/python/tf/config/experimental/
enable_op_determinism

Python random module
https://docs.python.org/3/library/random.html

NumPy random Generator
https://numpy.org/doc/stable/reference/random/generator.html

Scikit-learn decision trees
https://scikit-learn.org/stable/modules/tree.html

Scikit-learn linear models
https://scikit-learn.org/stable/modules/linear_model.html

Scikit-learn SVMs
https://scikit-learn.org/stable/modules/svm.html

OpenAI advanced usage and reproducible outputs
https://developers.openai.com/api/docs/guides/advanced-usage

OpenAI reproducible outputs with seed
https://developers.openai.com/cookbook/examples/
reproducible_outputs_with_the_seed_parameter

```
OpenAI web search tool
https://developers.openai.com/api/docs/guides/tools-web-search

Hugging Face generation strategies
https://huggingface.co/docs/transformers/en/generation_strategies

Hugging Face text generation class
https://huggingface.co/docs/transformers/en/main_classes/text_generation

MLflow tracking
https://mlflow.org/docs/latest/ml/tracking/

DVC versioning data and models
https://doc.dvc.org/example-scenarios/versioning-data-and-models

Dockerfile reference
https://docs.docker.com/reference/dockerfile/
```

---

1 4 5 7 13 18 22 https://docs.pytorch.org/docs/2.12/notes/randomness.html
https://docs.pytorch.org/docs/2.12/notes/randomness.html

2 20 Statistical Modeling: The Two Cultures (with comments and ...
https://projecteuclid.org/journals/statistical-science/volume-16/issue-3/Statistical-Modeling--The-Two-Cultures-with-comments-and-a/10.1214/ss/1009213726.full?utm_source=chatgpt.com

3 14 https://www.itl.nist.gov/div898/handbook/eda/section1/eda12.htm
https://www.itl.nist.gov/div898/handbook/eda/section1/eda12.htm

6 10 https://www.cs.toronto.edu/~rsalakhu/papers/srivastava14a.pdf
https://www.cs.toronto.edu/~rsalakhu/papers/srivastava14a.pdf

8 https://developers.openai.com/api/docs/guides/advanced-usage
https://developers.openai.com/api/docs/guides/advanced-usage

9 23 https://docs.python.org/3/library/random.html
https://docs.python.org/3/library/random.html

11 https://docs.pytorch.org/docs/2.12/notes/numerical_accuracy.html
https://docs.pytorch.org/docs/2.12/notes/numerical_accuracy.html

12 https://huggingface.co/docs/transformers/en/generation_strategies
https://huggingface.co/docs/transformers/en/generation_strategies

15 21 24 https://www.stat.berkeley.edu/~aldous/157/Papers/shmueli.pdf
https://www.stat.berkeley.edu/~aldous/157/Papers/shmueli.pdf

16 https://link.springer.com/article/10.1023/A%3A1010933404324
https://link.springer.com/article/10.1023/A%3A1010933404324

17 https://scikit-learn.org/stable/modules/tree.html
https://scikit-learn.org/stable/modules/tree.html

19 https://www.itl.nist.gov/div898/handbook/eda/section1/eda13.htm
https://www.itl.nist.gov/div898/handbook/eda/section1/eda13.htm